UNIVERSITY OF CRAIOVA
Faculty of Automation, Computers and Electronics · Department of Computers and Information Technology

Data Structures and Algorithms · Laboratory work

Binary heaps

min-heap and max-heap

A binary heap is a data structure that stores a complete binary tree in a plain array and gives O(1) access to the minimum (min-heap) or the maximum (max-heap). The lab covers insertion, root extraction, linear-time construction, key changes and heapsort, and then implements both variants in C++ from the same code base.

Prerequisites: functions and classes in C++, `std::vector`, recursion, O-notation. **Tools:** any C++11 or newer compiler, for example `g++ -std=c++11 -Wall -Wextra`.

Contents

1 Objectives . . . 2
2 Background . . . 2
2.1 Definition . . . 2
2.2 Array representation . . . 2
2.3 Cost of the operations . . . 3
3 Heap operations . . . 4
3.1 Insertion (sift-up) . . . 4
3.2 Root extraction (sift-down) . . . 4
3.3 Building a heap (build-heap) . . . 5
3.4 Changing and deleting a key . . . 6
3.5 Heapsort . . . 7
3.6 From a min-heap to a max-heap . . . 7
4 Lab work . . . 8
4.1 Implementation . . . 8
4.2 Program output . . . 10
5 Exercises . . . 10
6 Practice questions . . . 11
7 Review questions . . . 11
8 Source code . . . 12
9 References . . . 20
Appendix Answers to the practice questions . . . 21

1 Objectives

The objectives of this lab are:

- explaining the heap property and how a complete binary tree is stored in an array;
- tracing sift-up and sift-down on paper and estimating their cost;
- implementing in C++ insertion, root extraction, build-heap, key changes and heapsort;
- switching from a min-heap to a max-heap by changing the order relation;
- comparing the implementation with std::priority_queue and the heap algorithms of the C++ standard library;
- using heaps in applications: heapsort, top-k and the median of a stream.

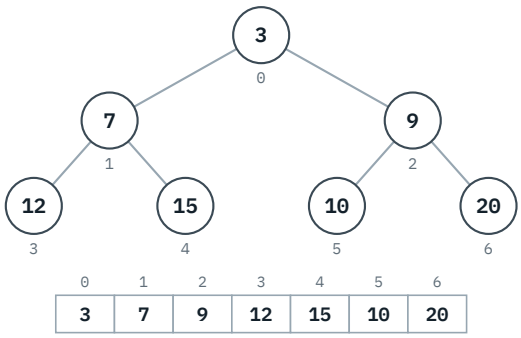
2 Background

2.1 Definition

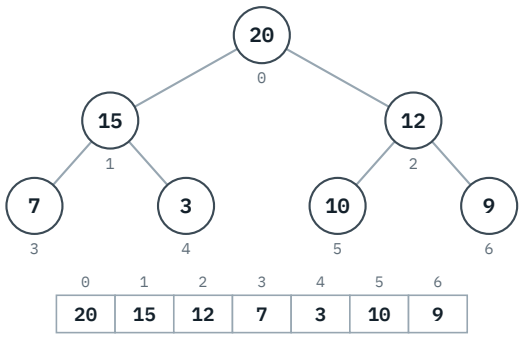
A **binary heap** is a binary tree that satisfies two conditions at once:

- **shape:** the tree is complete, i.e. every level is full except possibly the last one, which is filled from left to right;
- **order:** every node is ordered with respect to its children. In a **min-heap**, h[parent] ≤ h[child]; in a **max-heap**, h[parent] ≥ h[child].

The order property implies that the root holds the minimum (min-heap) or the maximum (max-heap). The property only links parents to children: siblings and nodes on different branches are not ordered, so a heap is not a sorted array. Figure 1 shows the same seven values arranged both ways.



(a) min-heap: the root is the minimum, 3



(b) max-heap: the root is the maximum, 20

Figure 1. The values 3, 7, 9, 10, 12, 15, 20 arranged as a min-heap and as a max-heap. Below each tree is the array that stores it, with the indices above the cells.

2.2 Array representation

Because the tree is complete, its nodes can be numbered level by level, left to right, starting with 0 at the root. Each node's number becomes its index in the array, so no pointers are needed: the parent and the children are computed (Table 1, Figure 2).

Table 1. Index relations in a heap with n elements, with 0-based indexing.

Relation	Index (0-based)
parent(i)	`⌊(i − 1) / 2⌋`
left(i)	`2i + 1`
right(i)	`2i + 2`
leaves	`⌊n/2⌋ … n − 1`
last internal node	`⌊n/2⌋ − 1`
tree height	$\lfloor \log_2 n \rfloor$

This lab uses 0-based indexing, as in C++. A heap with n nodes has height $\lfloor \log_2 n \rfloor$, so every path from the root to a leaf has at most $\lfloor \log_2 n \rfloor$ edges. This bound is what makes insertion and extraction cost O(log n).

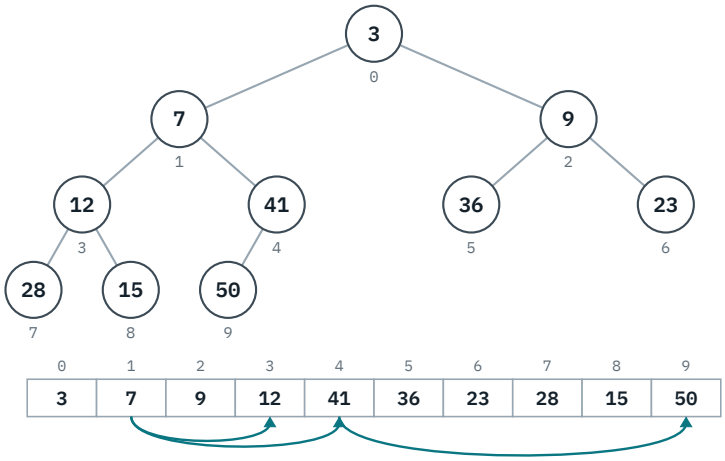


Figure 2. The min-heap used in the examples of section 3, stored in an array. The arcs below the array go from node 1 to its children, 2·1 + 1 = 3 and 2·1 + 2 = 4, and from node 4 to its only child, 9.

2.3 Cost of the operations

Table 2 summarises the cost of each operation. A heap fits whenever we repeatedly need the smallest (or largest) element of a changing set: priority queues, Dijkstra's and Prim's algorithms, process scheduling, merging k sorted lists.

Table 2. Cost of the operations on a binary heap with n elements.

Operation	Cost	Why
reading the root	`O(1)`	the minimum (maximum) is always in h[0]
insertion	`O(log n)`	sift-up climbs at most the height of the tree
root extraction	`O(log n)`	the last element moves to the root, then sift-down
changing a key	`O(log n)`	sift-up or sift-down, depending on the direction
deleting at a position	`O(log n)`	replace with the last element, then repair
build-heap	`O(n)`	most nodes sit near the leaves and move down only a little
heapsort	`O(n log n)`	n – 1 extractions, in place, with no extra memory
searching for an arbitrary value	`O(n)`	the heap order does not help searching

3 Heap operations

Every operation reduces to two repair procedures. **Sift-up** moves an element towards the root while it is “better” than its parent. **Sift-down**, also called *heapify*, moves an element towards the leaves by swapping it with the better of its children. To describe both heap types with a single algorithm we use the predicate BETTER(a, b): it is true when a < b in a min-heap and when a > b in a max-heap. “Better” therefore means smaller in a min-heap and larger in a max-heap.

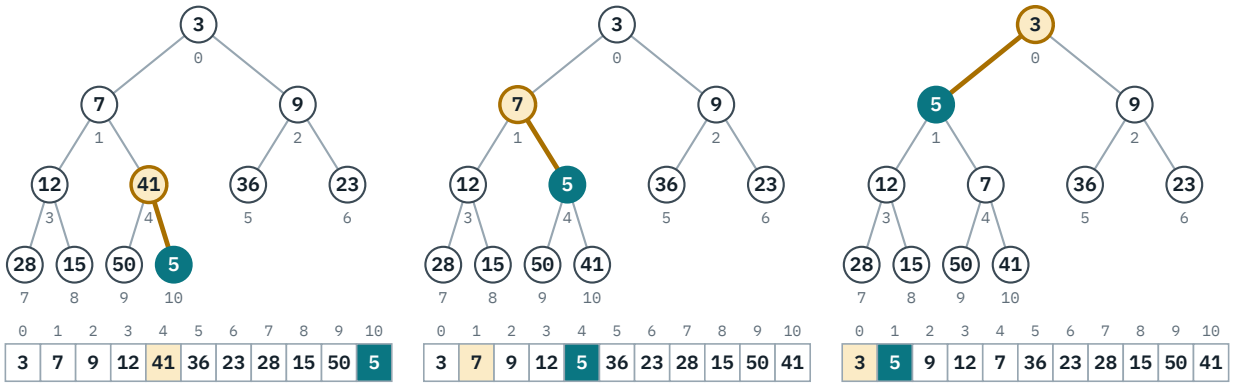
In the figures, the node being moved is filled in colour and the nodes it is compared with are yellow. The small numbers below the nodes are the array indices.

3.1 Insertion (sift-up)

The new element goes into the first free position, h[n], which keeps the tree complete. The order can now be violated only between the new element and its parent, so the element moves up: while it is better than its parent, the two are swapped. The process stops when the parent is at least as good or when the element reaches the root.

```
SIFT-UP(h, i)
  while i > 0 and BETTER(h[i], h[parent(i)])
      swap h[i] with h[parent(i)]
      i ← parent(i)

INSERT(h, n, x)
  h[n] ← x
  n ← n + 1
  SIFT-UP(h, n − 1)
```



(a) 5 at h[10]; 5 < 41 (b) after the swap; 5 < 7 (c) after the swap; 3 ≤ 5, stop

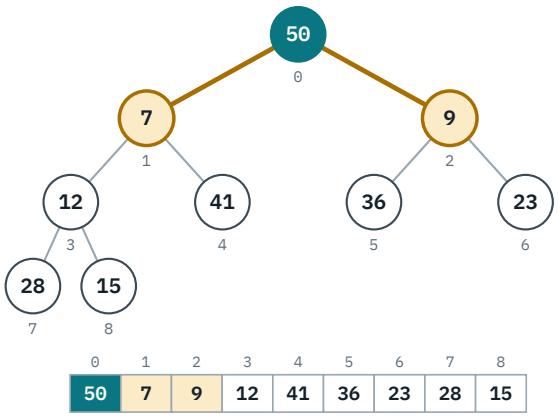
Figure 3. Inserting 5 into the min-heap of Figure 2: 2 swaps, fewer than the height of the tree. Result: `3, 5, 9, 12, 7, 36, 23, 28, 15, 50, 41`.

3.2 Root extraction (sift-down)

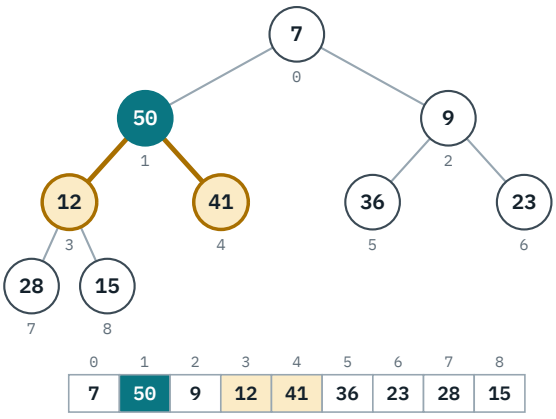
The root is returned as the result. Its place is taken by the last element, h[n − 1], and the size drops by one; the shape stays complete, but the new root is usually too “weak”. It moves down: the better of its children is chosen and, if that child is better than the current node, they are swapped. Choosing the better child matters: the child that moves up becomes the parent of its sibling, so it must be at least as good as the sibling.

```
SIFT-DOWN(h, n, i)
  repeat
      b ← i
      if left(i) < n and BETTER(h[left(i)], h[b]) then b ← left(i)
      if right(i) < n and BETTER(h[right(i)], h[b]) then b ← right(i)
      if b = i then stop
      swap h[i] with h[b]
      i ← b

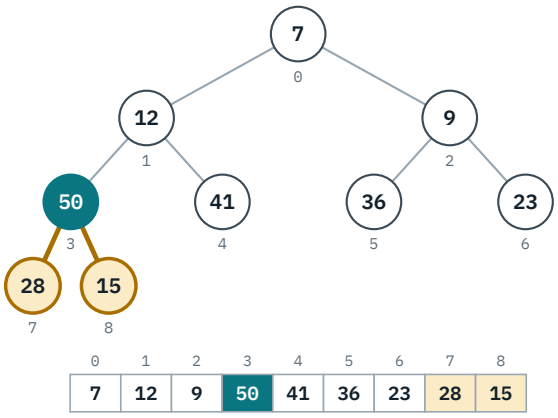
EXTRACT-ROOT(h, n)
  result ← h[0]
  n ← n − 1
  h[0] ← h[n]
  SIFT-DOWN(h, n, 0)
  return result
```



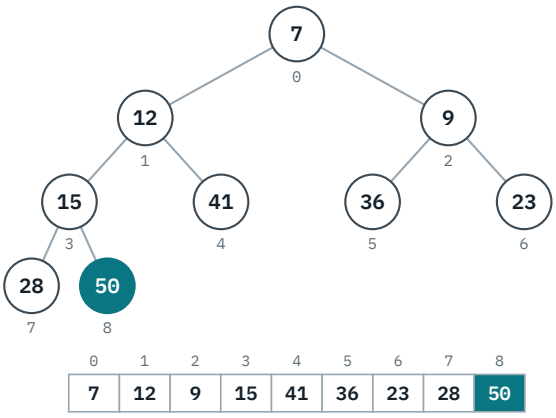
(a) 3 is removed, 50 moves to the root; children: 7 and 9



(b) 7 moves up; children of 50: 12 and 41



(c) 12 moves up; children of 50: 28 and 15



(d) 15 moves up; 50 is now a leaf, stop

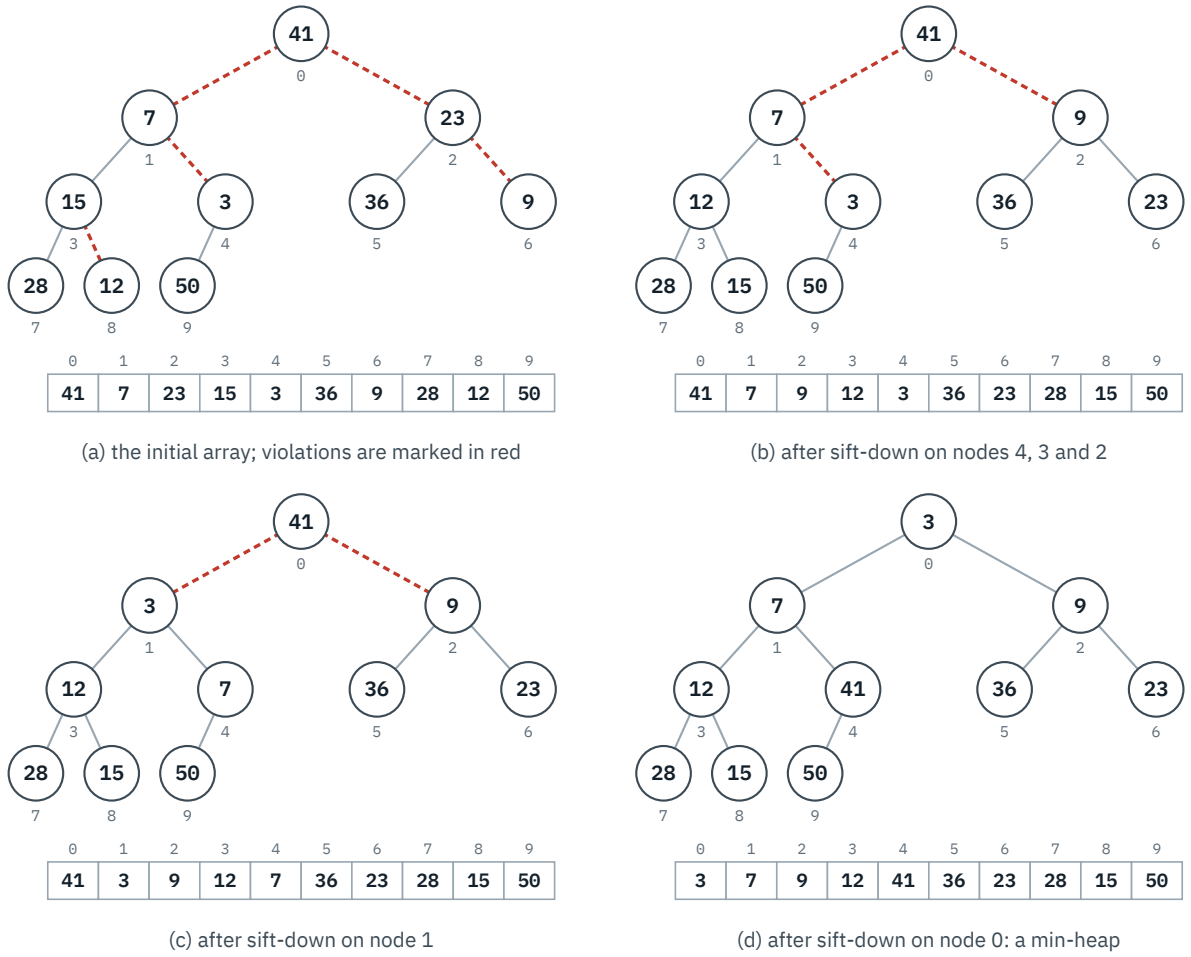
Figure 4. Extracting the minimum from the heap in Figure 2. The value returned is 3 and the remaining heap is `7, 12, 9, 15, 41, 36, 23, 28, 50`.

3.3 Building a heap (build-heap)

A heap of n elements can be obtained with n successive insertions, in O(n log n). The bottom-up method, due to Floyd, is faster: the given array is viewed directly as a complete tree, and sift-down is applied to every internal node, from the last one, $\lfloor n/2 \rfloor - 1$, back to the root. When node i is processed, its subtrees are already heaps, so sift-down can place it correctly. The leaves, which are half of the nodes, need no work at all.

The cost is O(n). A node at height k moves down at most k levels, and there are at most $\lceil n / 2^{k+1} \rceil$ nodes at height k. The total work is therefore at most $\Sigma\, k \cdot \lceil n / 2^{k+1} \rceil = O(n)$, because $\Sigma\, k / 2^{k+1} = 1$.

```
BUILD-HEAP(h, n)
  for i ← ⌊n/2⌋ − 1 down to 0
      SIFT-DOWN(h, n, i)
```



(a) the initial array; violations are marked in red

(b) after sift-down on nodes 4, 3 and 2

(c) after sift-down on node 1

(d) after sift-down on node 0: a min-heap

Figure 5. Build-heap for the array `41, 7, 23, 15, 3, 36, 9, 28, 12, 50`: 5 swaps. Nodes 9...5 are leaves, and sift-down on node 4 moves nothing, because 3 ≤ 50.

3.4 Changing and deleting a key

If the value at position i becomes better (smaller, in a min-heap), the order can only break with its parent, so sift-up is applied. This is the decrease-key operation used by Dijkstra's and Prim's algorithms. If the value becomes worse, the order can only break with its children, so sift-down is applied. In a max-heap, the symmetric operation is called increase-key.

Deleting the element at position i reduces to a key change: h[i] receives the value of the last element, the size drops by one, and the heap is repaired in whichever direction is needed.

```
CHANGE-KEY(h, n, i, x)
  old ← h[i]
  h[i] ← x
  if BETTER(x, old) then SIFT-UP(h, i)
  else SIFT-DOWN(h, n, i)

DELETE(h, n, i)
  n ← n − 1
  if i < n then CHANGE-KEY(h, n, i, h[n])
```

3.5 Heapsort

Heapsort sorts the array in place. First, build-heap turns the array into a heap. Then, at every step, the root, which is the best element of the heap zone, is swapped with the last element of that zone; the heap zone shrinks by one and a sift-down from the root repairs the remaining heap. With a max-heap the maxima settle from the end towards the front, so the result is ascending; with a min-heap it is descending. The algorithm runs in O(n log n) in every case and needs no extra memory, but it is not stable.

```
HEAPSORT(h, n)
  BUILD-HEAP(h, n)
  for end ← n − 1 down to 1
      swap h[0] with h[end]
      SIFT-DOWN(h, end, 0)
```

	0	1	2	3	4	5
after build-heap	10	5	8	4	1	3
step 1: 10 goes to h[5]	8	5	3	4	1	10
step 2: 8 goes to h[4]	5	4	3	1	8	10
step 3: 5 goes to h[3]	4	1	3	5	8	10
step 4: 4 goes to h[2]	3	1	4	5	8	10
step 5: 3 goes to h[1]	1	3	4	5	8	10

Figure 6. Heapsort with a max-heap on the array `4, 10, 3, 5, 1, 8`. The sorted zone, in colour, grows from right to left; each row shows the array after the swap and the sift-down.

3.6 From a min-heap to a max-heap

The two heap types differ only in the order relation. Table 3 lists what changes; the rest of the algorithms stays identical. The program in section 8 writes both versions explicitly, as pairs of methods: minSiftUp / maxSiftUp, minHeapify / maxHeapify, extractMin / extractMax.

Table 3. Correspondence between a min-heap and a max-heap.

Aspect	min-heap	max-heap
parent–child relation	h[p] ≤ h[c]	h[p] ≥ h[c]
root	the minimum	the maximum
BETTER(a, b)	a < b	a > b
child chosen by sift-down	the smaller one	the larger one
key change that moves up	decrease-key	increase-key
extraction	extractMin	extractMax
in the STL	`priority_queue<int, vector<int>, greater<int>>`	`priority_queue<int>`
in-place heapsort	descending order	ascending order
typical uses	Dijkstra, Prim; the k largest values	the k smallest values; max-priority queue

4 Lab work

4.1 Implementation

The functions below implement the operations of section 3 in C++, on a `std::vector<int>`. The BETTER predicate of the pseudocode is the function `better`, so switching from a min-heap to a max-heap means changing a single line. Insertion uses `push_back` and extraction uses `pop_back`. The main program builds a heap from the array in Figure 5, inserts 5 and extracts the root.

```
#include <iostream>
#include <utility>
#include <vector>

// Change only this function to switch between a min-heap and a max-heap
bool better(int a, int b) { return a < b; }   // min-heap: parent <= children

int parent(int i) { return (i - 1) / 2; }
int left(int i)   { return 2 * i + 1; }
int right(int i)  { return 2 * i + 2; }

// Move h[i] up while it is better than its parent
void siftUp(std::vector<int>& h, int i)
{
    while (i > 0 && better(h[i], h[parent(i)])) {
        std::swap(h[i], h[parent(i)]);
        i = parent(i);
    }
}

// Move h[i] down within the heap made of the first n elements
void siftDown(std::vector<int>& h, int n, int i)
{
    for (;;) {
        int l = left(i), r = right(i), b = i;
        if (l < n && better(h[l], h[b])) b = l;
        if (r < n && better(h[r], h[b])) b = r;
        if (b == i) return;                  // h[i] has reached its place
```

```
        std::swap(h[i], h[b]);
        i = b;
    }
}

void heapInsert(std::vector<int>& h, int x)
{
    h.push_back(x);                          // first free position
    siftUp(h, static_cast<int>(h.size()) - 1);
}

// Assumes the heap is not empty
int heapExtract(std::vector<int>& h)
{
    int top = h[0];
    h[0] = h.back();                         // the last element moves to the root
    h.pop_back();
    siftDown(h, static_cast<int>(h.size()), 0);
    return top;
}

// Build-heap in O(n): siftDown on the internal nodes, from the last one up
void buildHeap(std::vector<int>& h)
{
    int n = static_cast<int>(h.size());
    for (int i = n / 2 - 1; i >= 0; i--)
        siftDown(h, n, i);
}

bool isHeap(const std::vector<int>& h)
{
    for (int i = 1; i < static_cast<int>(h.size()); i++)
        if (better(h[i], h[parent(i)])) return false;
    return true;
}

void printArray(const std::vector<int>& h)
{
    for (int x : h) std::cout << x << ' ';
    std::cout << '\n';
}

int main()
{
    std::vector<int> h = {41, 7, 23, 15, 3, 36, 9, 28, 12, 50};

    buildHeap(h);
    printArray(h);
    std::cout << "isHeap: " << isHeap(h) << '\n';

    heapInsert(h, 5);
    printArray(h);

    std::cout << "extracted: " << heapExtract(h) << '\n';
    printArray(h);
    return 0;
}
```

4.2 Program output

Compile and run the program. Table 4 shows what it prints with `better` set for a min-heap and, after changing the order relation, with `better` set for a max-heap.

Table 4. Program output, for each heap type.

min-heap `return a < b;`

```
3 7 9 12 41 36 23 28 15 50
isHeap: 1
3 5 9 12 7 36 23 28 15 50 41
extracted: 3
5 7 9 12 41 36 23 28 15 50
```

max-heap `return a > b;`

```
50 41 36 28 7 23 9 15 12 3
isHeap: 1
50 41 36 28 7 23 9 15 12 3 5
extracted: 50
41 28 36 15 7 23 9 5 12 3
```

5 Exercises

The exercises build on the implementation in section 4.

1. **One code base, both heaps.** Change only the `better` function and check the max-heap output against Table 4. Then turn the code into a class template, `template <class Better> class Heap`, with `Heap<std::less<int>>` for a min-heap and `Heap<std::greater<int>>` for a max-heap, so that one program can hold both heaps.
2. **Build-heap versus repeated insertion.** Add a global comparison counter to `better`. For $n = 10^3$, 10^4 and 10^5 random values, count the comparisons made by `buildHeap` and by n calls to `heapInsert`. Report the ratio comparisons / n in both cases and explain the difference.
3. **Heapsort.** Implement `void heapSort(std::vector<int>& h)`. A max-heap must give ascending order and a min-heap descending order. Test it on an already sorted array, on a reverse-sorted one and on one with many equal values, and compare the result with `std::sort`.
4. **Comparison with the STL.** The standard library offers the same operations: `std::make_heap`, `std::push_heap`, `std::pop_heap`, `std::sort_heap` and `std::priority_queue` [4, 5]. For random arrays, check the result of `buildHeap` with `std::is_heap` and the extraction order with a `std::priority_queue`. Mind the convention: in the STL the comparator says which element has the lower priority, so `std::priority_queue<int>` is a max-heap, and a min-heap needs `std::priority_queue<int, std::vector<int>, std::greater<int>>`.
5. **Top-k.** Read n numbers and print the k largest using a min-heap of size k, in O(n log k) time and O(k) memory. You may use your own heap or `std::priority_queue<int, std::vector<int>, std::greater<int>>`. The reverse variant: the k smallest values, with a max-heap.
6. **Bonus: running median.** Keep the lower half of the values in a max-heap and the upper half in a min-heap. After each number read, rebalance so that the sizes differ by at most one, and print the median.

6 Practice questions

Answer the questions on paper, drawing the tree where it helps. Indexing starts at 0. The answers are in the Appendix.

1. Which of the following arrays are heaps? Give the type: min, max or neither.
   (a) `2, 5, 3, 9, 6, 4, 8`
   (b) `4, 6, 9, 8, 5, 10, 12`
   (c) `50, 30, 40, 10, 20, 35, 25`
   (d) `1, 2, 3, 4, 5, 6, 7`
2. Compute: (a) the index of the parent of h[37]; (b) the indices of the children of h[12]; (c) the level of h[20], with the root on level 0; (d) the height of a heap with 100 nodes; (e) the index of the first leaf in a heap with 45 nodes; (f) the index at which build-heap starts for n = 45.
3. Insert 4 and then 1 into the min-heap `3, 7, 9, 12, 41, 36, 23, 28, 15, 50`. Write the array after each insertion and the number of swaps.
4. Extract the maximum twice from the max-heap `50, 41, 36, 28, 7, 23, 9, 15, 12, 3`. Write the array after each extraction.
5. Use build-heap to make a min-heap and, separately, a max-heap from the array `9, 4, 7, 1, 8, 2, 6, 3, 5`. Write the resulting array and the number of swaps in each case.
6. In the min-heap `3, 7, 9, 12, 41, 36, 23, 28, 15, 50`: (a) h[6] becomes 2; (b) starting again from the original heap, h[1] becomes 60. Write the array after each change.
7. Run heapsort with a max-heap on the array `6, 2, 9, 4, 7`. Write the array after build-heap and after each step of the sorting loop.
8. In a min-heap with 50 distinct keys, at which indices can the maximum be? How many comparisons are needed, in the worst case, to find it?
9. Show that an array sorted in ascending order is always a min-heap. Is the converse true? Give a counterexample.
10. What is the largest number of swaps when inserting into a heap that already holds 1000 elements? And when extracting the root from the same heap?

7 Review questions

1. Why does build-heap start at index $\lfloor n/2 \rfloor - 1$ and not at $n - 1$?
2. Why does sift-down swap the node with the better of its children and not with the first child that breaks the order?
3. Why does build-heap cost O(n), even though it makes up to n/2 calls to sift-down that each cost O(log n)?
4. Why is heapsort not stable? Give an example with two equal keys.
5. What changes in the formulas of Table 1 if the root is stored in h[1] instead of h[0]?
6. Why is std::priority_queue<int> a max-heap, even though its default comparator is std::less<int>?

8 Source code

The complete menu-driven program starts from the MinHeap class in [3], with the fields `harr`, `capacity` and `heap_size` and the operations `insertKey`, `decreaseKey`, `extractMin`, `deleteKey` and `MinHeapify`. The `Heap` class keeps its elements in a `std::vector<int>`, so it has no fixed capacity, and extends the code in three ways: every min-heap method has a mirrored max-heap twin; a 13-option menu chooses the heap type at start-up and can switch it later; the tree is drawn in the console after every operation, and the step-by-step mode (option 13) prints every swap.

Build: `g++ -std=c++11 -Wall -Wextra -o heap_menu heap_menu_en.cpp`.

```cpp
// heap_menu_en.cpp - menu-driven binary heap: min-heap or max-heap
// Data Structures and Algorithms lab, University of Craiova
//
// Starts from the MinHeap class on GeeksforGeeks (harr, capacity, heap_size,
// parent/left/right, insertKey, decreaseKey, extractMin, deleteKey, MinHeapify),
// https://www.geeksforgeeks.org/dsa/binary-heap/, and adds:
//   - a mirrored max-heap version: every min... method has a max... twin
//   - an interactive menu, with the heap type chosen at start-up and switchable later
//   - a console drawing of the tree and a "step by step" mode that shows
//     every swap made by sift-up and heapify
// The elements live in a std::vector, so the capacity is no longer fixed.
//
// Build: g++ -std=c++11 -Wall -Wextra -o heap_menu_en heap_menu_en.cpp

#include <algorithm>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <utility>
#include <vector>

const int DRAW_LEVELS = 6;   // taller trees are printed level by level

enum class HeapType { Min, Max };

class Heap {
public:
    explicit Heap(HeapType t) : type(t) {}

    HeapType getType() const { return type; }
    int size() const { return static_cast<int>(harr.size()); }
    int at(int i) const { return harr[i]; }
    bool getTrace() const { return trace; }
    void setTrace(bool on) { trace = on; }

    void insertKey(int k);
    bool getRoot(int& out) const;
    bool extractMin(int& out);
    bool extractMax(int& out);
    bool extractRoot(int& out);
    bool decreaseKey(int i, int newVal);
    bool increaseKey(int i, int newVal);
    bool changeKey(int i, int newVal);
    bool deleteKey(int i, int& out);
    void buildHeap();
    void load(const std::vector<int>& v);
    void switchType();
    bool isHeap() const;
    std::vector<int> sorted() const;

    void printArray() const;
```

```
    void printTree() const;

private:
    std::vector<int> harr;   // the heap elements
    HeapType type;           // HeapType::Min or HeapType::Max
    bool trace = false;      // true = print every swap

    static int parent(int i) { return (i - 1) / 2; }
    static int left(int i)   { return 2 * i + 1; }
    static int right(int i)  { return 2 * i + 2; }

    void minSiftUp(int i);
    void maxSiftUp(int i);
    void minHeapify(int i, int n);
    void maxHeapify(int i, int n);
    void siftUp(int i);
    void heapify(int i, int n);
    void traceSwap(int i, int j) const;
};

// ---------------------------------------------------------------------------
// Helpers
// ---------------------------------------------------------------------------
static const char* typeName(HeapType t)
{
    return t == HeapType::Min ? "min-heap" : "max-heap";
}

static const char* rootName(HeapType t)
{
    return t == HeapType::Min ? "minimum" : "maximum";
}

// Number of characters x takes when printed (including the minus sign).
static int numWidth(int x)
{
    return static_cast<int>(std::to_string(x).size());
}

// Called right after a swap between positions i and j.
void Heap::traceSwap(int i, int j) const
{
    if (!trace) return;
    std::cout << "  swap h[" << i << "] = " << harr[j]
              << " with h[" << j << "] = " << harr[i] << '\n';
    printTree();
}

// ---------------------------------------------------------------------------
// Moving up (sift-up): the min / max pair
// ---------------------------------------------------------------------------
void Heap::minSiftUp(int i)
{
    while (i > 0 && harr[i] < harr[parent(i)]) {
        std::swap(harr[i], harr[parent(i)]);
        traceSwap(i, parent(i));
        i = parent(i);
    }
}

void Heap::maxSiftUp(int i)
{
    while (i > 0 && harr[i] > harr[parent(i)]) {
        std::swap(harr[i], harr[parent(i)]);
        traceSwap(i, parent(i));
        i = parent(i);
    }
}
```

```
121
122 // ------------------------------------------------------------------------
123 // Moving down (heapify / sift-down): the min / max pair
124 // Works on the first n elements and assumes the subtrees of i
125 // are already heaps.
126 // ------------------------------------------------------------------------
127 void Heap::minHeapify(int i, int n)
128 {
129     int l = left(i), r = right(i), smallest = i;
130     if (l < n && harr[l] < harr[smallest]) smallest = l;
131     if (r < n && harr[r] < harr[smallest]) smallest = r;
132     if (smallest != i) {
133         std::swap(harr[i], harr[smallest]);
134         traceSwap(i, smallest);
135         minHeapify(smallest, n);
136     }
137 }
138
139 void Heap::maxHeapify(int i, int n)
140 {
141     int l = left(i), r = right(i), largest = i;
142     if (l < n && harr[l] > harr[largest]) largest = l;
143     if (r < n && harr[r] > harr[largest]) largest = r;
144     if (largest != i) {
145         std::swap(harr[i], harr[largest]);
146         traceSwap(i, largest);
147         maxHeapify(largest, n);
148     }
149 }
150
151 // Pick the version that matches the heap type.
152 void Heap::siftUp(int i)
153 {
154     if (type == HeapType::Min) minSiftUp(i); else maxSiftUp(i);
155 }
156
157 void Heap::heapify(int i, int n)
158 {
159     if (type == HeapType::Min) minHeapify(i, n); else maxHeapify(i, n);
160 }
161
162 // ------------------------------------------------------------------------
163 // Operations
164 // ------------------------------------------------------------------------
165 void Heap::insertKey(int k)
166 {
167     harr.push_back(k);          // first free position; the vector grows by itself
168     siftUp(size() - 1);
169 }
170
171 // The root is the minimum (min-heap) or the maximum (max-heap).
172 bool Heap::getRoot(int& out) const
173 {
174     if (harr.empty()) return false;
175     out = harr[0];
176     return true;
177 }
178
179 bool Heap::extractMin(int& out)
180 {
181     if (harr.empty()) return false;
182     out = harr[0];
183     harr[0] = harr.back();      // the last element moves to the root
184     harr.pop_back();
185     minHeapify(0, size());
186     return true;
187 }
188
```

```cpp
bool Heap::extractMax(int& out)
{
    if (harr.empty()) return false;
    out = harr[0];
    harr[0] = harr.back();
    harr.pop_back();
    maxHeapify(0, size());
    return true;
}

bool Heap::extractRoot(int& out)
{
    return type == HeapType::Min ? extractMin(out) : extractMax(out);
}

// Min-heap: the key may only decrease, so the element may only move up.
bool Heap::decreaseKey(int i, int newVal)
{
    if (i < 0 || i >= size() || newVal > harr[i]) return false;
    harr[i] = newVal;
    minSiftUp(i);
    return true;
}

// Max-heap: the key may only increase, so the element may only move up.
bool Heap::increaseKey(int i, int newVal)
{
    if (i < 0 || i >= size() || newVal < harr[i]) return false;
    harr[i] = newVal;
    maxSiftUp(i);
    return true;
}

// Change in either direction: towards the root with decreaseKey / increaseKey,
// otherwise the element moves down with heapify.
bool Heap::changeKey(int i, int newVal)
{
    if (i < 0 || i >= size()) return false;
    if (type == HeapType::Min) {
        if (newVal <= harr[i]) return decreaseKey(i, newVal);
        harr[i] = newVal;
        minHeapify(i, size());
    } else {
        if (newVal >= harr[i]) return increaseKey(i, newVal);
        harr[i] = newVal;
        maxHeapify(i, size());
    }
    return true;
}

// Delete the element at position i: replace it with the last element
// and restore the order in whichever direction is needed.
// Classic alternative: decreaseKey(i, INT_MIN) + extractMin for a min-heap,
// or increaseKey(i, INT_MAX) + extractMax for a max-heap.
bool Heap::deleteKey(int i, int& out)
{
    if (i < 0 || i >= size()) return false;
    out = harr[i];
    int last = harr.back();
    harr.pop_back();
    if (i < size()) changeKey(i, last);
    return true;
}

// Build-heap in O(n): heapify the internal nodes, from the last one up to the root.
void Heap::buildHeap()
{
    for (int i = size() / 2 - 1; i >= 0; i--)
```

```
        heapify(i, size());
}

void Heap::load(const std::vector<int>& v)
{
    harr = v;
    buildHeap();
}

void Heap::switchType()
{
    type = (type == HeapType::Min) ? HeapType::Max : HeapType::Min;
    buildHeap();
}

bool Heap::isHeap() const
{
    for (int i = 1; i < size(); i++) {
        int p = harr[parent(i)], c = harr[i];
        if (type == HeapType::Min ? p > c : p < c) return false;
    }
    return true;
}

// In-place heapsort on a copy, so the heap in the menu stays unchanged.
// min-heap -> descending order, max-heap -> ascending order.
std::vector<int> Heap::sorted() const
{
    Heap copy = *this;
    copy.trace = false;
    for (int end = copy.size() - 1; end > 0; end--) {
        std::swap(copy.harr[0], copy.harr[end]);   // the root goes to its final place
        copy.heapify(0, end);
    }
    return copy.harr;
}

// ---------------------------------------------------------------------------
// Output
// ---------------------------------------------------------------------------
void Heap::printArray() const
{
    if (harr.empty()) { std::cout << "The heap is empty.\n"; return; }
    int w = numWidth(size() - 1);
    for (int x : harr) w = std::max(w, numWidth(x));
    w++;
    std::cout << "  index:";
    for (int i = 0; i < size(); i++) std::cout << std::setw(w) << i;
    std::cout << "\n  value:";
    for (int x : harr) std::cout << std::setw(w) << x;
    std::cout << '\n';
}

// Print the line without its trailing spaces.
static void printTrimmed(const std::string& s)
{
    std::size_t end = s.find_last_not_of(' ');
    std::cout << "  " << (end == std::string::npos ? "" : s.substr(0, end + 1)) << '\n';
}

// Tree drawing: each node is centred in its own "slice" of the width.
// Level d has 2^d slices; a leaf on the last level gets cw characters.
void Heap::printTree() const
{
    int n = size(), height = 0, cw = 4;
    if (n == 0) { std::cout << "  (empty heap)\n"; return; }
    while ((2 << height) - 1 < n) height++;          // height = floor(log2 n)
    for (int x : harr) cw = std::max(cw, numWidth(x) + 2);
```

```
325
326     if (height >= DRAW_LEVELS) {                    // too wide for a console
327         for (int d = 0, first = 0; first < n; d++, first = 2 * first + 1) {
328             std::cout << "  level " << d << ":";
329             for (int i = first; i < n && i <= 2 * first; i++)
330                 std::cout << ' ' << harr[i];
331             std::cout << '\n';
332         }
333         return;
334     }
335
336     int total = (1 << height) * cw;
337     for (int d = 0; d <= height; d++) {
338         int slots = 1 << d, sw = total / slots, first = slots - 1;
339
340         std::string line(total, ' ');
341         for (int k = 0; k < slots && first + k < n; k++) {
342             std::string s = std::to_string(harr[first + k]);
343             int len = static_cast<int>(s.size());
344             line.replace(k * sw + (sw - len) / 2, s.size(), s);
345         }
346         printTrimmed(line);
347
348         if (d < height) {                           // edges towards the children
349             std::string edges(total, ' ');
350             for (int k = 0; k < slots && first + k < n; k++) {
351                 int i = first + k;
352                 if (left(i) < n)  edges[k * sw + 3 * sw / 8] = '/';
353                 if (right(i) < n) edges[k * sw + 5 * sw / 8] = '\\';
354             }
355             printTrimmed(edges);
356         }
357     }
358 }
359
360 // ---------------------------------------------------------------------------
361 // Keyboard input
362 // ---------------------------------------------------------------------------
363 static bool readLine(const std::string& prompt, std::string& line)
364 {
365     std::cout << prompt << std::flush;
366     if (!std::getline(std::cin, line)) return false;    // EOF
367     if (!line.empty() && line.back() == '\r') line.pop_back();
368     return true;
369 }
370
371 // Repeats until it gets a valid integer. Returns false on EOF.
372 static bool readInt(const std::string& prompt, int& out)
373 {
374     std::string line;
375     while (readLine(prompt, line)) {
376         std::istringstream in(line);
377         int v;
378         char extra;
379         if (in >> v && !(in >> extra)) { out = v; return true; }
380         std::cout << "  Invalid value, please enter an integer.\n";
381     }
382     return false;
383 }
384
385 // Reads numbers separated by spaces or commas from one line.
386 // Returns how many it read, or -1 on EOF.
387 static int readList(const std::string& prompt, std::vector<int>& v)
388 {
389     std::string line;
390     if (!readLine(prompt, line)) return -1;
391     for (char& c : line)
392         if (c == ',' || c == ';') c = ' ';
```

```
    std::istringstream in(line);
    v.clear();
    int x;
    for (;;) {
        if (in >> x) v.push_back(x);
        else if (in.eof()) break;
        else { in.clear(); in.ignore(1); }          // skip one invalid character
    }
    return static_cast<int>(v.size());
}

// ---------------------------------------------------------------------------
// Menu
// ---------------------------------------------------------------------------
static void printMenu(const Heap& h)
{
    bool mn = (h.getType() == HeapType::Min);
    std::cout << "\n=== " << typeName(h.getType()) << " | " << h.size()
              << " elements | step by step: " << (h.getTrace() ? "ON" : "OFF")
              << " ===\n"
              << " 1. Insert a value\n"
              << " 2. Show the root (" << rootName(h.getType()) << ")\n"
              << " 3. Extract the root (" << (mn ? "extractMin" : "extractMax") << ")\n"
              << " 4. " << (mn ? "Decrease" : "Increase") << " the key at an index ("
              << (mn ? "decreaseKey" : "increaseKey") << ")\n"
              << " 5. Change the key at an index, in either direction\n"
              << " 6. Delete the element at an index\n"
              << " 7. Build a heap from a list of values (buildHeap)\n"
              << " 8. Heapsort (" << (mn ? "descending" : "ascending") << " order)\n"
              << " 9. Show the array\n"
              << "10. Show the tree\n"
              << "11. Check the heap property\n"
              << "12. Switch to " << (mn ? "max-heap" : "min-heap")
              << " (rebuilds the heap)\n"
              << "13. Turn step-by-step mode " << (h.getTrace() ? "off" : "on") << "\n"
              << " 0. Exit\n";
}

static bool validIndex(const Heap& h, int i)
{
    if (i >= 0 && i < h.size()) return true;
    if (h.size() == 0)
        std::cout << "The heap is empty.\n";
    else
        std::cout << "Invalid index: it must be between 0 and "
                  << h.size() - 1 << ".\n";
    return false;
}

int main()
{
    int choice, t, i, x;
    std::vector<int> v;
    bool running = true;

    std::cout << "Binary heap - DSA lab\n"
              << " 1. min-heap (parent <= children, the root is the minimum)\n"
              << " 2. max-heap (parent >= children, the root is the maximum)\n";
    do {
        if (!readInt("Choose the type (1 or 2): ", t)) return 0;
    } while (t != 1 && t != 2);

    Heap h(t == 1 ? HeapType::Min : HeapType::Max);

    while (running) {
        printMenu(h);
        if (!readInt("Option: ", choice)) break;

```

```
        switch (choice) {
        case 0:
            running = false;
            break;

        case 1:
            if (!readInt("Value: ", x)) { running = false; break; }
            h.insertKey(x);
            std::cout << "Inserted " << x << ". Result:\n";
            h.printTree();
            break;

        case 2:
            if (h.getRoot(x))
                std::cout << "The root (" << rootName(h.getType()) << ") is "
                          << x << ".\n";
            else
                std::cout << "The heap is empty.\n";
            break;

        case 3:
            if (h.extractRoot(x)) {
                std::cout << "Extracted " << x << ". Result:\n";
                h.printTree();
            } else {
                std::cout << "The heap is empty.\n";
            }
            break;

        case 4: {
            if (!readInt("Index: ", i)) { running = false; break; }
            if (!validIndex(h, i)) break;
            if (!readInt("New value: ", x)) { running = false; break; }
            bool mn = (h.getType() == HeapType::Min);
            if (mn ? h.decreaseKey(i, x) : h.increaseKey(i, x)) {
                std::cout << "Result:\n";
                h.printTree();
            } else {
                std::cout << "In a " << typeName(h.getType()) << ", "
                          << (mn ? "decreaseKey" : "increaseKey") << " needs a value "
                          << (mn ? "smaller" : "greater") << " than or equal to "
                          << h.at(i) << ".\nUse option 5 instead.\n";
            }
            break;
        }

        case 5:
            if (!readInt("Index: ", i)) { running = false; break; }
            if (!validIndex(h, i)) break;
            if (!readInt("New value: ", x)) { running = false; break; }
            h.changeKey(i, x);
            std::cout << "Result:\n";
            h.printTree();
            break;

        case 6:
            if (!readInt("Index: ", i)) { running = false; break; }
            if (!validIndex(h, i)) break;
            h.deleteKey(i, x);
            std::cout << "Deleted " << x << ". Result:\n";
            h.printTree();
            break;

        case 7: {
            int n = readList("Values (e.g. 41 7 23 15 3 36 9 28 12 50): ", v);
            if (n < 0) { running = false; break; }
            if (n == 0) { std::cout << "No values were read.\n"; break; }
            h.load(v);
```

```
            std::cout << "Built a " << typeName(h.getType()) << " with " << n
                      << " elements:\n";
            h.printArray();
            h.printTree();
            break;
        }

        case 8:
            if (h.size() == 0) { std::cout << "The heap is empty.\n"; break; }
            std::cout << "Sorted "
                      << (h.getType() == HeapType::Min ? "descending" : "ascending")
                      << ":";
            for (int y : h.sorted()) std::cout << ' ' << y;
            std::cout << '\n';
            break;

        case 9:
            h.printArray();
            break;

        case 10:
            h.printTree();
            break;

        case 11:
            std::cout << "The array " << (h.isHeap() ? "satisfies" : "does NOT satisfy")
                      << " the " << typeName(h.getType()) << " property.\n";
            break;

        case 12:
            h.switchType();
            std::cout << "Switched to " << typeName(h.getType()) << ". Result:\n";
            h.printTree();
            break;

        case 13:
            h.setTrace(!h.getTrace());
            std::cout << "Step-by-step mode is "
                      << (h.getTrace() ? "on" : "off") << ".\n";
            break;

        default:
            std::cout << "Unknown option.\n";
        }
    }

    std::cout << "Goodbye!\n";
    return 0;
}
```

9 References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022, chapter 6, “Heapsort”.

[2] R. Sedgewick, K. Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011, section 2.4, “Priority Queues”.

[3] GeeksforGeeks, “Binary Heap”, https://www.geeksforgeeks.org/dsa/binary-heap/.

[4] cppreference.com, “std::priority_queue”, https://en.cppreference.com/w/cpp/container/priority_queue.

[5] cppreference.com, “std::make_heap”, https://en.cppreference.com/w/cpp/algorithm/make_heap.

Appendix Answers to the practice questions

1. (a) min-heap
   (b) neither: h[1] = 6 > h[4] = 5 breaks the min-heap property, and h[0] = 4 < h[1] = 6 breaks the max-heap property
   (c) max-heap
   (d) min-heap (the array is sorted ascending)
2. (a) $\lfloor 36/2 \rfloor = 18$; (b) $2 \cdot 12 + 1 = 25$ and $2 \cdot 12 + 2 = 26$; (c) $\lfloor \log_2 21 \rfloor = 4$; (d) $\lfloor \log_2 100 \rfloor = 6$; (e) $\lfloor 45/2 \rfloor = 22$; (f) $\lfloor 45/2 \rfloor - 1 = 21$.
3. After inserting 4: `3, 4, 9, 12, 7, 36, 23, 28, 15, 50, 41` (2 swaps).
   After inserting 1: `1, 4, 3, 12, 7, 9, 23, 28, 15, 50, 41, 36` (3 swaps).
4. The first extraction returns 50: `41, 28, 36, 15, 7, 23, 9, 3, 12`.
   The second extraction returns 41: `36, 28, 23, 15, 7, 12, 9, 3`.
5. min-heap: `1, 3, 2, 4, 8, 7, 6, 9, 5` (6 swaps).
   max-heap: `9, 8, 7, 5, 4, 2, 6, 3, 1` (2 swaps).
6. (a) `2, 7, 3, 12, 41, 36, 9, 28, 15, 50`: 2 moves all the way up to the root.
   (b) `3, 12, 9, 15, 41, 36, 23, 28, 60, 50`: 60 moves down to a leaf.
7. after build-heap: `9, 7, 6, 4, 2`
   step 1: `7, 4, 6, 2, 9`
   step 2: `6, 4, 2, 7, 9`
   step 3: `4, 2, 6, 7, 9`
   step 4: `2, 4, 6, 7, 9`
8. The maximum is in a leaf, so at one of the indices 25…49: 25 candidates, i.e. 24 comparisons.
9. A parent always has a smaller index than its child, so in an ascending array h[parent] ≤ h[child]. The converse is false: 1, 3, 2 is a min-heap but is not sorted.
10. The new element lands at index 1000, on level $\lfloor \log_2 1001 \rfloor = 9$, so at most 9 swaps. After an extraction 999 elements remain, with height $\lfloor \log_2 999 \rfloor = 9$: again at most 9 swaps, but up to 18 comparisons.